

The ufrgscca, and associated, Packages

Version 2.13a

(extended documentation)

Alceu Frigeri*

July 2026

Abstract

This bundled is aimed at producing undergraduate students final work/report at UFRGS/EE (Engineering School at the Federal University of Rio Grande do Sul), closely following ABNT rules (Brazilian Association for Technical Norms). It is composed of a main class, *ufrgscca*, and a set of auxiliary packages, some of which can be used independently.

Contents

1	Introduction	2
1.1	Current Version	2
2	<i>ufrgscca</i> Class	2
2.1	Class Options	3
2.2	Class Declared Commands	4
2.3	Class Known Hooks	4
3	<i>ufrgscca-abnt</i> Package	5
3.1	Package Options	5
3.2	Commands	5
3.3	Environments	7
3.4	Tabular New Columns	7
3.5	enumitem Extra Keys	8
4	<i>ufrgscca-core</i> Package	10
4.1	Core Commands	11
4.2	Core Auxiliary Commands	12
4.3	Core Specific Commands	13
5	<i>ufrgscca-cover</i> Package	14
5.1	Defined Commands	14
6	<i>ufrgscca-forms</i> Package	15
6.1	Package Options	15
6.2	Forms Defined Commands	15

*<https://github.com/alceu-frigeri/ufrgscca>

7	<u>ufrgscca-lists</u> Package	16
7.1	Environment	16
7.2	Declared Commands	16
8	<u>ufrgscca-coord</u> Package (extended documentation)	17
9	<u>ufrgscca-ppc</u> Package (extended documentation)	21
9.1	Package Options	22
9.2	Defined Commands	22
9.3	Environments	24
10	<u>ufrgscca-curr</u> Package (extended documentation)	24

1 INTRODUCTION

ABNT rules can be quite challenging at times (read: bibliography style/references) and sometimes just odd (line spacing, front matter, page layout), nevertheless it is a *Brazilian Standard* for typography whose students at UFRGS should grow cherished to follow.

In short, as of version 2.13a the bundle is composed of a class, *ufrgscca* (based on *report* class), which pre-loads, as needed: *ufrgscca-abnt*, *ufrgscca-core*, *ufrgscca-cover* and *ufrgscca-lists*. The packages *ufrgscca-forms*, *ufrgscca-curr*, *ufrgscca-coord* and *ufrgscca-ppc* need to be loaded explicitly, thought.

N.B.: This bundle requires a quite recent L^AT_EX2e kernel, at least as recent as June 2022, which allows to declare package options using the *key=value* system and declare commands with `\NewDocumentCommand`, out-of-the-box.

Version 2.0: Starting with Version 2.0, this bundle has been fully re-written with *Expl3* and *starray*.

Version 1.12a: In case you need the old version, for some odd *expl3* compatibility issue, you can find it at <https://github.com/alceu-frigeri/ufrgscca/releases/tag/v1.12a>

1.1 Current Version

For the sake of the “maintainer’s sanity”, since this is a bundle, all files are saved with the same version (bundle version: 2.13a)

2 UFRGSCCA CLASS

The following packages are always pre-loaded: *etex*, *etoolbox*, *lmodern*, *fontenc* (*T1*), *inputenc* (*utf8*), *silence* and *ufrgscca-abnt*, *ufrgscca-cover*, *ufrgscca-core* and *hyperref* and (if it exists) a *local.tex* file.

Other, auxiliary, packages are also pre-loaded, depending on the class options used, and finally *report* class (the exception being in case of the *article* class option).

Being based on the report class, all report class options are valid, in addition to the ones below.

2.1 Class Options

<code>tocdepth</code>	use: <code>tocdepth=number</code> , whereas $\langle \text{number} \rangle$ indicates the deepest sectioning to appears in the Table of Contents (0 being the top section, which is <code>\chapter</code> for report based classes, 1 being <code>\section</code> , and so on.) The default is 3 (<code>\subsubsection</code>).
<code>secdepth</code>	use: <code>secdepth=number</code> , whereas $\langle \text{number} \rangle$ indicates the deepest sectioning to be numbered. (0 being the top section, which is <code>\chapter</code> for report based classes, 1 being <code>\section</code> , and so on.) The default is 4 (<code>\paragraph</code>).
<code>english</code>	the default language being Portuguese, this option changes locale to English.
<code>brazilian</code>	in some rare cases (to be further investigated) babel seems to get confused about which language is active, this “shouldn’t be necessary” but one can explicitly tell babel to use THIS language (which should, otherwise, be the default one).
<code>relnum</code>	by default, figures, tables, etc. are numbered as a continuous series. With this switch, those lists are reset at each chapter, e.g. Figure 5.1 instead of Figure 23.
<code>openright</code>	in case of printed material, this assures that a <code>\chapter</code> always starts at an odd page, which is relevant in case of printing out (double sided) the document.
<code>oneside</code>	in case the document will be printed in single side sheets, otherwise it’s assumed a two-sided printing.
<code>strict-abnt</code>	to assure asymmetric margins, as defined by ABNT: inner ones greater than outer ones, which matters if you are going to print the doc and make a book of it, but makes it odd to look at in a computer screen, reason by which the current default setting is for symmetric margins (same text width).
<code>pretextontoc</code>	“pre-text” elements, like “list of...” will be inserted in the “table of contents”.
<code>yearonly</code>	Approval page, at it’s bottom, will display the years only (instead of the default mouth year construct).
<code>timesroman</code>	will set the default font to Roman (using the obsolete <code>mathptmx</code> package, based on a free replacement of the proprietary Times New Roman (by Microsoft) and Times Roman (By Adobe)) instead of the default Latin Modern Roman font. As a side effect, the package <code>microtype</code> isn’t loaded (can’t be used), resulting in a sub-optimal overall layout. NB. The alternative (newer and maintained) packages <code>newtxtext</code> / <code>newtxmath</code> can’t be used due to some packages incompatibilities.
<code>repeatfields</code>	in case of authors with multiple publications, their names will be repeated for each entry. In the default setting the author’s name is written only in the first entry, and replaced by underscores in the other entries.
<code>xlists</code>	this will load the <code>ufrgscca-lists</code> package, for the definition of new floats/lists.
<code>xpacks</code>	this will load a series of packages, which can be handy when writing Engineering reports: <code>relsize</code> , <code>keyval</code> , <code>graphicx</code> , <code>mathtools</code> , <code>mathrsfs</code> , <code>amsfonts</code> , <code>amssymb</code> , <code>empheq</code> , <code>amsthm</code> , <code>extarrows</code> , <code>mathfixs</code> , <code>bigdelim</code> , <code>circuitikz</code> and <code>steimenz</code> and tikz libraries: <code>fit</code> , <code>math</code> , <code>calc</code> , <code>shapes.geometry</code> , <code>shapes.misc</code> , <code>shapes.multipart</code> , <code>graphs</code> , <code>3d</code> , <code>positioning</code> , <code>shadows</code> and <code>babel</code> . One is advised to look after each package documentation (ctan.org) for further information.
<code>chapternopagenum</code>	to suppress the page numbers at chapters begin.

<code>nomicrotype</code>	in some rare cases, <i>microtype</i> might hurt page layout, this allows the suppression of <i>microtype</i> .
<code>showframes</code>	for layout proof only, it will draw frames around each page main parts.
<code>showlabels</code>	it will put a reference mark in each label created, and print out it's name.
<code>nofontwarning</code>	in case of <i>ufrgscca-ppc</i> is loaded, it will suppress some font related warnings.
<code>nolocal</code>	this will suppress the loading of any local.tex file, which would, otherwise, be loaded.
<code>article</code>	this will load the class <i>article</i> instead of <i>report</i> , it's meant to document the class itself.
<code>nogeometry</code>	the package <i>geometry</i> won't be loaded. In case one wants to fully customize the page geometry
<code>oldrenews</code>	Some, deprecated, renews will be in effect: <code>\maketitle</code> , <code>\author</code> <code>\begin{abstract}</code> . For backwards compatibility only.
<code>texlive</code>	this is a reserved key, in case some workaround for texlive is needed.
<code>overleaf</code>	this is a reserved key, in case some workaround for overleaf is needed.
<code>miktex</code>	this is a reserved key, in case some workaround for miktex is needed.

2.2 Class Declared Commands

<code>\autonameref</code>	<code>\autonameref [<sep>] {<label>} [<spc>]</code>
<code>\annexref</code>	<code>\annexref {<label>}</code>
<code>\autoannexref</code>	<code>\autoannexref [<sep>] {<label>} [<spc>]</code>

The *hyperref* package, sometimes, gets the `\autoref` name wrong (when referencing an annex), the `\annexref {<label>}` will assure the correct annex name is used.

`\autonameref {<label>}` produces an entry of the form '`\autoref {<label>} <sep> \nameref {<label>} <spc>`'

`\autoannexref {<label>}` produces an entry of the form '`\annexref {<label>} <sep> \nameref {<label>} <spc>`'

The default `<sep>` being a comma, and the default `<spc>` being empty space.

2.3 Class Known Hooks

<code>\miktexHack</code>	<code>\miktexHack</code>
<code>\overleafHack</code>	<code>\overleafHack</code>
<code>\livetexHack</code>	<code>\livetexHack</code>

Case some workaround is needed due an unexpected error (when upgrading packages/TeXsystem) the class "knows" about those three hooks. They will be executed if, and only if, they are user defined and the corresponding package option is used, i.e., for example, for the hook `\miktexHack` to be used/called by the class *ufrgscca*, one has to: a) define it and b) use the class option `miktex`.

3 UFRGSCCA-ABNT PACKAGE

This package is the one that sets the page layout (using *geometry*, *titlesec* and *titletoc*) and adjusts the main float environments (figure, tables, captions). It can be used as a stand alone package, regardless of the underlying class.

The following packages are always pre-loaded: *babel*, *csquotes*, *geometry*, *appendix*, *titlesec*, *titletoc*, *enumitem*, *chngctr*, *caption*, *biblatex*, *microtype*, *array*, *nicematrix*, *contour* and *soul*.

Take note that *biblatex* is loaded with the **biber** option, to correctly handle ABNT biography style.

3.1 *Package Options*

<code>strict-abnt</code>	to assure asymmetric margins, as defined by ABNT: inner ones greater than outer ones, which matters if you are going to print the doc and make a book of it, but makes it odd to look at in a computer screen, reason by which the current default setting is for symmetric margins (same text width).
<code>chapternopagenum</code>	to suppress the page numbers at chapters begin.
<code>relnum</code>	by default, figures, tables, etc. are numbered as a continuous series. With this switch, those lists are reset at each chapter, e.g. Figure 5.1 instead of Figure 23.
<code>repeatfields</code>	in case of authors with multiple publications, their names will be repeated for each entry. In the default setting the author's name is written only in the first entry, and replaced by underscores in the other entries.
<code>yearonly</code>	In some cover pages (like the ones for TCC) the bottom of the approval's page, will only displays the year (instead of the default mouth year construction).
<code>nomicrotype</code>	in some rare cases, <i>microtype</i> might hurt page layout, this allows the suppression of <i>microtype</i> .
<code>showframes</code>	for layout proof only, it will draw frames around each page main parts.
<code>showlabels</code>	it will put a reference mark in each label created, and print out it's name.
<code>tocdepth</code>	use: <code>tocdepth = <number></code> , whereas <code><number></code> indicates the deepest sectioning to appears in the Table of Contents (0 being the top section, which is <code>\chapter</code> for report based classes, 1 being <code>\section</code> , and so on.) The default value being 3 (<code>\subsubsection</code>).
<code>secdepth</code>	use: <code>secdepth = <number></code> , whereas <code><number></code> indicates the deepest sectioning to be numbered. (0 being the top section, which is <code>\chapter</code> for report based classes, 1 being <code>\section</code> , and so on.) The default value being 4 (<code>\paragraph</code>).

3.2 *Commands*

<code>\mainkeyword</code>	<code>\mainkeyword {<keyword>}</code>
<code>\otherkeyword</code>	<code>\otherkeyword {<keyword>}</code>

new: 2023/11/18

These command can be invoked many times, it will construct a list of keywords to be used when printing out the (main/other)abstract environment.

Note: the old `\keyword {<keyword>}` gets defined, as an alias to `\mainkeyword`, in case one use the *oldrenews* class option is used.

<code>\sourcecitation</code>	<code>\sourcecitation {<source>}</code>
<code>\note</code>	<code>\note {<text>}</code>

When describing floating elements (like figure, tables, circuits) one always has to cite the source of it, and in some cases it might be necessary to add a special note. Those assure uniformity when doing that.

<code>\nonum</code>	<code>\nonum\chapter {<chap.title>}</code>
<code>\notoc</code>	<code>\nonum\section {<sec.title>}</code>
	<code>\notoc\chapter {<chap.title>}</code>
	<code>\notoc\section {<sec.title>}</code>

In some cases, it might be necessary to create a numberless chapters or sections. Those two commands can be used as a *prefix* to any sectioning command. Whilst `\nonum` will just suppress the sectioning number, the `\notoc` will also suppress it from the table of contents.

LaTeXCode:

```
\nonum\chapter{some title} %this one will appear in the toc
\notoc\section{some other title} %this won't even appear in the toc
```

<code>\tightul</code>	<code>\tightul {<text>}</code>
-----------------------	--------------------------------------

This will *underline* a short text, take note that `<text>` ‘can’t be broken’ (think paragraph justification), which can lead to *text overflows* and bad justification.

LaTeXCode:

LaTeXResult:

<code>\tightul{Some text example}%</code>	<u>Some text example</u>
---	--------------------------

<code>\NewChapListEnv</code>	<code>\NewChapListEnv {<envname>} {<displayname>}</code>
------------------------------	--

Creates a *chapter like* lists, e.g. ‘List of Symbols’ or ‘List of acronyms’. With it, a new environment is created, `<envname>`, with an associated ‘numberless’ chapter name `<displayname>`. The newly created environment will implement a *description* like environment (thanks to *enumitem*) with an optional and a mandatory argument (see below).

LaTeXCode:

```
\def\listabbrvname{Lista de Abreviaturas}
\NewChapListEnv{listofabbrv}{\listabbrvname} % this is the actual code
used in ufrgscga-abnt.sty
```

<code>\pubdate</code>	<code>\pubdate [<day>] {<month>} {<year>}</code>
<code>\today</code>	<code>\today</code>
<code>\monthname</code>	<code>\monthname</code>

updated: 2023/11/30

`\pubdate` sets the publication date. If not called by the user it *defaults* to current month / year. `\today` returns the current *locale* date, whilst `\monthname` returns the *locale* name of the current month.

N.B.: If the package option `oldrenews` is used, the command `\date` will be redefined as an alias to `\pubdate`.

3.3 Environments

<code>mainabstract</code>	<code>\begin{mainabstract} [⟨lang⟩] {⟨keywords⟩}...\end{mainabstract}</code>
---------------------------	--

new: 2023/11/18

`mainabstract` is defined as a numberless chapter based on the current locale (default: Portuguese), at the end of it the keywords list created with `\mainkeyword` will be added.
L^AT_EX Code:

```
\mainkeyword{a keyword}
\mainkeyword{another keyword}
\begin{mainabstract} some short summary of things\ldots
\end{mainabstract}
```

<code>otherabstract</code>	<code>\begin{otherabstract} [⟨lang⟩] {⟨...⟩}\end{otherabstract}</code>
----------------------------	--

updated: 2023/11/18

This is the environment to create an abstract in a language other than the default one. The default value for `⟨lang⟩` is english, and it can be any value that `babel` understands. A keywords list created with `\otherkeyword` will be added at the end of it.
L^AT_EX Code:

```
\otherkeyword{a keyword}
\otherkeyword{another keyword}
\begin{otherabstract}[english] some short summary of things\ldots
\end{otherabstract}
```

<code>listofabbrv</code>	<code>\begin{listofabbrv} [⟨enum-opt⟩] {⟨ABBRV⟩}...\end{listofabbrv}</code>
<code>listofsymbols</code>	<code>\begin{listofsymbols} [⟨enum-opt⟩] {⟨SYMB⟩}...\end{listofsymbols}</code>

Both environments create a description like list preceded by a numberless (`\nonum`) chapter. `⟨enum-opt⟩` is any `enumitem` list valid key. Whereas `⟨ABBRV⟩` / `⟨SYMB⟩` are just the ‘biggest’ abbreviation/symbol to be used as a tab reference.

<code>appendix</code>	<code>\begin{appendix} \end{appendix}</code>
<code>annex</code>	<code>\begin{annex} \end{annex}</code>

Those two environments start the appendices and annex chapters (using locale). Chapters are alphabetic *numbered* (starting at A).

3.4 Tabular New Columns

Thanks to `array` some new columns types are defined:

P `P{⟨width⟩}` Normal text, ragged left.

- B *B{<width>}* Bold text, ragged left.
C *C{<width>}* Normal text, centered.
R *R{<width>}* Normal text, ragged left.
L *L{<width>}* Normal text, ragged right.
J *J{<width>}* Normal text, justified.

3.5 *enumitem* Extra Keys

Besides the *default* keys defined by the *enumitem* package a few others are defined for author's convenience:

- `ppc, tcc` *ppc* and *tcc* are alias of each other, and just assure that lists indentation will be the same as paragraphs default.
`parindent` with *parindent*, the list number/mark is aligned with paragraph indentation.
`noindent` *noindent* removes the label indentation.

LaTeXCode:

LaTeXResult:

```
\begin{enumerate}[tcc]
  \item some A
  \item some B
\end{enumerate}
\begin{enumerate}[tcc,parindent]
  \item some A
  \item some B
\end{enumerate}
\begin{enumerate}[parindent]
  \item some A
  \item some B
\end{enumerate}
\begin{enumerate}[noindent]
  \item some A
  \item some B
\end{enumerate}
```

1. some A
2. some B
1. some A
2. some B
1. some A
2. some B
1. some A
2. some B

New paragraph, for reference.

New paragraph, for reference.

- `tight` allows for very tight lists (no indentation) to be used, for instance, inside quotes. N.B. don't use it in normal paragraph mode, otherwise the labels will spill outside the default text window.
`miditemsep` *miditemsep* halves items separation, as an alternative to *noitemsep* from *enumitem*

LaTeXCode:

LaTeXResult:

<code>\begin{enumerate}[tcc]</code>	
<code>\item some A</code>	1. some A
<code>\item some B</code>	2. some B
<code>\end{enumerate}</code>	
<code>\begin{enumerate}[tcc,miditemsep]</code>	
<code>\item some A</code>	1. some A
<code>\item some B</code>	2. some B
<code>\end{enumerate}</code>	
<code>\begin{enumerate}[tcc,noitemsep]</code>	
<code>\item some A</code>	1. some A
<code>\item some B</code>	2. some B
<code>\end{enumerate}</code>	

bullet for simple itemized lists, it will replace the default black dot by an ‘open bullet’

LaTeXCode:

LaTeXResult:

<code>\begin{itemize}[tcc,miditemsep]</code>	
<code>\item some A</code>	• some A
<code>\item some B</code>	• some B
<code>\item some C</code>	• some C
<code>\end{itemize}</code>	
<code>\begin{itemize}[</code>	
<code>tcc,bullet,miditemsep]</code>	◦ some A
<code>\item some A</code>	◦ some B
<code>\item some B</code>	◦ some C
<code>\item some C</code>	
<code>\end{itemize}</code>	

arabic That’s the *default* enumerate style. Arabic numbers, starting at 1, followed by a dot.

arabic) Label will be constructed as number followed by a parenthesis.

(arabic) Label will be enclosed by parenthesis.

arabic* (for secondary lists) Label will be constructed by the label of the outer list, this item number and a final dot.

arabic*) (for secondary lists) Label will be constructed by the label of the outer list, this item number and a final parenthesis.

roman This and below keys are the same as the arabic ones, but using lower case roman numbers.

roman) lower case roman number, followed by a parenthesis.

(roman) enclosed by parenthesis.

roman* preceding one followed by roman number and a final dot.

roman*) same, followed by a final parenthesis.

Roman This and below keys are the same as the arabic ones, but using upper case roman numbers.

Roman) upper case roman number, followed by a parenthesis.

(Roman) enclosed by parenthesis.

Roman* preceding one followed by roman number and a final dot.

Roman*) same, followed by a final parenthesis.

- `alpha` This and below keys are the same as the arabic ones, but using lower case alpha numbers.
- `alpha)` lower case alpha number, followed by a parenthesis.
- `(alpha)` enclosed by parenthesis.
- `alpha*` preceding one followed by alpha number and a final dot.
- `alpha*)` same, followed by a final parenthesis.
- `Alpha` This and below keys are the same as the arabic ones, but using upper case alpha numbers.
- `Alpha)` upper case roman number, followed by a parenthesis.
- `(Alpha)` enclosed by parenthesis.
- `Alpha*` preceding one followed by roman number and a final dot.
- `Alpha*)` same, followed by a final parenthesis.

L^AT_EX Code:

L^AT_EX Result:

```
\begin{enumerate}[tcc,roman]
  \item some A
  \item some B
  \item some C
\end{enumerate}
\begin{enumerate}[tcc,Roman]
  \item some A
  \item some B
  \begin{enumerate}[tcc,alpha*]
    \item some A
    \item some B
    \item some C
  \end{enumerate}
  \item some C
\end{enumerate}
\begin{enumerate}[tcc,arabic]
  \item some A
  \item some B
  \begin{enumerate}[tcc,roman*)]
    \item some A
    \item some B
    \item some C
  \end{enumerate}
  \item some C
\end{enumerate}
```

- i. some A
- ii. some B
- iii. some C
- I. some A
- II. some B
 - II.a. some A
 - II.b. some B
 - II.c. some C
- III. some C
 - 1. some A
 - 2. some B
 - 2.i) some A
 - 2.ii) some B
 - 2.iii) some C
 - 3. some C

4 UFRGSCCA-CORE PACKAGE

The *ufrgscca-core* package defines a set of commands for student's and activity's related info. It is needed by most/all of the bundled packages.

All data is stored in two main *starray* defined as follow:

Activity's Structure Definition:

```
{
  name , acronym ,
  coord . struct
  {
    name , title ,
    article , Article , narticle , Narticle , carticle , Carticle ,
  } ,
  calendar . struct
  {
    date , week , event ,
  } ,
  chkmarked , chkunmarked , chkref
}
```

Student's Structure Definition:

```
{
  first , last , name , Nproc , ID , email , worktitle ,
    article , Article , narticle , Narticle , carticle , Carticle ,
  remarks , checklist , brief , reason ,
  board-local , board-date , board-time , gradeavg , grade ,
  flag-graded , %%% IF gradeavg AND finalgrade already calculated (or defined)
  flag-exam ,
  flag-ff ,
  flag-dismiss , %%% IF it was the 1st semester.
  flag-other , %%% "other list", placeholder for 'none of the above' lists.
  flag-newpage , %%% if it should go in a new page (board)
  flag-distinctboard , %%% if advisor isn't in the board
  flag-approved ,
  flag-coadvisor ,
  advisor . struct {
    first , last , name , institution , title , email , phone ,
    article , Article , narticle , Narticle , carticle , Carticle ,
    assessment
  } ,
  coadvisor . struct {
    first , last , name , institution , title , email , phone ,
    article , Article , narticle , Narticle , carticle , Carticle ,
    reason
  } ,
  reviewer . struct {
    first , last , name , institution , title , email , phone ,
    article , Article , narticle , Narticle , carticle , Carticle ,
    pointA , pointB , pointC , pointD , grade , gradetype , flag-examreview ,
  } ,
  altreviewer . struct {
    first , last , name , institution , title , email , phone ,
    article , Article , narticle , Narticle , carticle , Carticle ,
  } ,
  internship . struct {
    company , field , start , end , length ,
  } ,
  tutor . struct {
    first , last , name , title , email , phone ,
    article , Article , narticle , Narticle , carticle , Carticle ,
  } ,
  supervisor . struct {
    first , last , name , register , title , office , email , phone ,
    article , Article , narticle , Narticle , carticle , Carticle ,
  } ,
}
```

4.1 Core Commands

`\NewActivity` `\NewActivity {⟨act-hash⟩}`

new: 2023/11/18

This will create a new 'activity'. Predefined ones being: `course`, `tccI`, `tccII`, `internship` and `internship-opt`.

Note: This will create a *starray*, the $\langle \text{act-hash} \rangle$ being it's $\langle \text{hash} \rangle$.

<code>\ActivitySet</code>	<code>\ActivitySet [$\langle \text{act-hash} \rangle$] {$\langle \text{acronym} \rangle$}{$\langle \text{name} \rangle$}</code>
<code>\ActivitySetCoordTitle</code>	<code>\ActivitySetCoordTitle [$\langle \text{act-hash} \rangle$] {$\langle \text{title} \rangle$}</code>
<code>\ActivitySetCoord</code>	<code>\ActivitySetCoord [$\langle \text{act-hash} \rangle$] {$\langle \text{name} \rangle$}[$\langle \text{gender} \rangle$]</code>

new: 2023/11/18

These will set an Activity many fields. $\langle \text{acronym} \rangle$ and $\langle \text{name} \rangle$ being the short (acronym) and long name of an activity. $\langle \text{title} \rangle$ is the coordinator formal title, and so on.

<code>\ActivitySelect</code>	<code>\ActivitySelect {$\langle \text{act-hash} \rangle$}</code>
<code>\Activity</code>	<code>\Activity [$\langle \text{act-hash} \rangle$] {$\langle \text{act-field} \rangle$}</code>
<code>\ActivityCoord</code>	<code>\ActivityCoord [$\langle \text{act-hash} \rangle$] {$\langle \text{coord-field} \rangle$}</code>

new: 2023/11/18

`\ActivitySelect` just sets $\langle \text{act-hash} \rangle$ as the current activity (set's the *starray* iter). `\Activity` and `\ActivityCoord` gets the corresponding field. Possible values for $\langle \text{act-field} \rangle$ are: *name* and *acronym*. Possible values for $\langle \text{coord-field} \rangle$ are: *name*, *title*, *article*, *Article*, *narticle*, *Narticle*, *carticle* and *Carticle*, as defined by *Activity's structure* (see 4).

4.2 Core Auxiliary Commands

<code>\studentselect</code>	<code>\studentselect {$\langle \text{student-hash} \rangle$}</code>
<code>\studentReviewerSelect</code>	<code>\studentReviewerSelect {$\langle \text{reviewer-hash} \rangle$}</code>

new: 2023/11/18

Selects a student or reviewer based on theirs hash.

<code>\DataFields</code>	<code>\DataFields {$\langle \text{starray-ref} \rangle$}{$\langle \text{field} \rangle$}</code>
<code>\eDataSet</code>	<code>\eDataSet [$\langle \text{act-hash} \rangle$] {$\langle \text{starray-ref} \rangle$}</code>
<code>\eDataFields</code>	<code>\eDataFields {$\langle \text{field} \rangle$}</code>
<code>\DataGet</code>	<code>\DataFields {$\langle \text{starray-ref} \rangle$}{$\langle \text{field} \rangle$}{$\langle \text{tl-var} \rangle$}</code>

new: 2023/11/18
updated: 2024/04/18

These are, respectively, `\starray_get_prop:nn`, `\starray_term_parser:n` and `\starray_parsed_get_prop:n` and `\starray_get_prop:nnN` from *starray*. One can reference/get any field from the main *starray* defined structures: *student* and *activity* as defined at chapter 4.

<code>\studentiterate</code>	<code>\studentiterate {$\langle \text{code} \rangle$}</code>
<code>\studentadvisoriterate</code>	<code>\studentadvisoriterate {$\langle \text{code} \rangle$}</code>

new: 2023/11/18
updated: 2023/12/02

These are `\starray_iterate_over:nn` from *starray*. $\langle \text{code} \rangle$ will be executed for every defined student, `\studentiterate` or student's advisor, `\studentadvisoriterate`.

<code>\ActivityCalendarIterate</code>	<code>\ActivityCalendarIterate {$\langle \text{code} \rangle$}</code>
---------------------------------------	--

new: 2023/11/29

This is `\starray_iterate_over:nn` from *starray*. $\langle \text{code} \rangle$ will be executed for every defined calendar item.

4.3 Core Specific Commands

The following commands are more or less self-explanatory, $\langle ID \rangle$ is the student's university ID. $\langle Nproc \rangle$ is the process/request number. $\langle gender \rangle$ can be either 'm' or 'f'.

<code>\student</code>	<code>\student</code> [$\langle student-hash \rangle$] [$\langle last \rangle$] [$\langle first \rangle$] [$\langle gender \rangle$]
<code>\studentinfo</code>	<code>\studentinfo</code> [$\langle Nproc \rangle$] [$\langle ID \rangle$] [$\langle email \rangle$]

updated: 2023/11/18

N.B.: If the package option `oldrenews` is used, the command `\author` and `\authorinfo` will be redefined as an alias to `\student` and `\studentinfo`.

<code>\workbrief</code>	<code>\workbrief</code> [$\langle work-summary \rangle$]
<code>\advisorreview</code>	<code>\advisorreview</code> [$\langle advisor's-review \rangle$]
<code>\coadvisorreason</code>	<code>\coadvisorreason</code> [$\langle reason-for-a-coadvisor \rangle$]
<code>\workchange</code>	<code>\workchange</code> [$\langle reason-for-the-change \rangle$]

new: 2023/11/18

Those commands are only of use when using *ufrgscca-forms*. `\workbrief` sets the work initial summary, `\coadvisorreason` sets the justification for having a co-advisor, `\advisorreview` sets the advisor's review, `\workchange` sets the reason for the work's theme change.

<code>\advisor</code>	<code>\advisor</code> [$\langle title \rangle$] [$\langle last \rangle$] [$\langle first \rangle$] [$\langle gender \rangle$]
<code>\advisorinfo</code>	<code>\advisorinfo</code> [$\langle Institut \rangle$] [$\langle title-info \rangle$] [$\langle email \rangle$] [$\langle phone \rangle$]

<code>\coadvisor</code>	<code>\coadvisor</code> [$\langle title \rangle$] [$\langle last \rangle$] [$\langle first \rangle$] [$\langle gender \rangle$]
<code>\coadvisorinfo</code>	<code>\coadvisorinfo</code> [$\langle Institut \rangle$] [$\langle title-info \rangle$] [$\langle email \rangle$] [$\langle phone \rangle$]

<code>\distinctboard</code>	<code>\distinctboard</code>
-----------------------------	-----------------------------

new: 2023/11/18

For the rare case in which the advisor won't take part in the examiner's board.

<code>\examiner</code>	<code>\examiner</code> [$\langle title \rangle$] [$\langle last \rangle$] [$\langle first \rangle$] [$\langle gender \rangle$]
<code>\examinerinfo</code>	<code>\examinerinfo</code> [$\langle Institut \rangle$] [$\langle title-info \rangle$] [$\langle email \rangle$] [$\langle phone \rangle$]

<code>\altexaminer</code>	<code>\altexaminer</code> [$\langle title \rangle$] [$\langle last \rangle$] [$\langle first \rangle$] [$\langle gender \rangle$]
<code>\altexaminerinfo</code>	<code>\altexaminerinfo</code> [$\langle Institut \rangle$] [$\langle title-info \rangle$] [$\langle email \rangle$] [$\langle phone \rangle$]

<code>\internship</code>	<code>\internship</code> [$\langle company \rangle$] [$\langle field \rangle$] [$\langle start \rangle$] [$\langle end \rangle$] [$\langle length \rangle$]
--------------------------	---

new: 2023/11/18

<code>\tutor</code>	<code>\tutor</code> [$\langle title \rangle$] [$\langle last \rangle$] [$\langle first \rangle$] [$\langle gender \rangle$]
<code>\tutorinfo</code>	<code>\tutorinfo</code> [$\langle Institut \rangle$] [$\langle title-info \rangle$] [$\langle email \rangle$] [$\langle phone \rangle$]

<u><code>\supervisor</code></u>	<code>\supervisor</code> [<code><title></code>] [<code><last></code>]{ <code><first></code> }[<code><gender></code>]
<u><code>\supervisorinfo</code></u>	<code>\supervisorinfo</code> [<code><register></code>]{ <code><office></code> }{ <code><email></code> }{ <code><phone></code> }

N.B.: The commands `\advisor`, `\coadvisor`, `\examiner` and `\altexaminer` are meant to be used in a 'final work' doc. The Macros `\internship`, `\tutor` and `\supervisor` in case of an internship report.

5 UFRGSCCA-COVER PACKAGE

This package is the one that sets the front pages, depending on the kind of 'report' being generated.

5.1 *Defined Commands*

<u><code>\MakeCoverPages</code></u>	<code>\MakeCoverPages</code> [<code><type></code>]
<u>new: 2023/11/18</u>	This is the main command, which will typeset the front matter, from the information already given. <code><type></code> sets the 'kind' of cover pages to be generated. Currently, it can be one of:
<code>tccI</code>	Generate 3 pages, a first cover one, a second with work's description and a third last one with work's approval for TCC-I
<code>tccII</code>	Generate 3 pages, a first cover one, a second with work's description and a third last one with work's approval for TCC-II
<code>internship</code>	Generate 2 pages, a first cover one and a second with work's approval for internship report
<code>internship-opt</code>	Generate 2 pages, a first cover one and a second with work's approval for optional internship report
<code>class-report</code>	Generate 1 cover page

N.B.: If the package option `oldrenews` is used, the command `\maketitle` will be redefined as an alias to `\MakeCoverPages`.

<u><code>\location</code></u>	<code>\location</code> [<code><city></code>]{ <code><state></code> }
	To redefine the default values of <code><city></code> and <code><state></code> (Porto Alegre and RS).

<u><code>\class</code></u>	<code>\class</code> [<code><code></code>]{ <code><name></code> }
<u>new: 2023/11/18</u>	To set the class code and name, for the cover page, in case of a class report.

<u><code>\SetCoverFields</code></u>	<code>\SetCoverFields</code> [<code><type></code>]{ <code><field></code> }{ <code><value></code> }
<u>new: 2023/11/18</u>	This allows to redefine the aforementioned <code><types></code> and create new types of cover pages. <code><field></code> is one of:
<code>clist</code>	this defines which kind, and order, of pages will be generated. Possible values are: <code>cover</code> , <code>desc</code> and <code>approval</code> .

<code>top</code>	This will be the common top matter used.
<code>students</code>	How students names, authors, will be presented
<code>title</code>	The title to be used
<code>bottom</code>	The bottom of the cover page.
<code>text-descpage</code>	The text presented in the desc page.
<code>advisor-descpage</code>	Advisor's matter.
<code>bottom-descpage</code>	The bottom of the desc page.
<code>text-approvalpage</code>	The text presented in the approval page.
<code>advisor-approvalpage</code>	Advisor's matter in the approval page.
<code>bottom-approvalpage</code>	The bottom of the approval page.

6 UFRGSCCA-FORMS PACKAGE

This package defines just two user commands to create forms needed at UFRGS/EE.

6.1 *Package Options*

<code>noextrbookmarks</code>	By default, each form will have an associated PDF bookmark, this option will suppress them.
------------------------------	---

6.2 *Forms Defined Commands*

<code>\tcforms</code>	<code>\tcforms</code> {<formslist>}																						
<code>\tcmptyforms</code>	<code>\tcmptyforms</code> {<formslist>}																						
<u>updated: 2023/05/29</u>	The command <code>\tcforms</code> will generate the many forms (<formslist>) using the information from <i>local.tex</i>, whilst <code>\tcmptyforms</code> will generate said forms with 'blanks' (to be filled by hand, for instance). <formslist> is a csv list of: <table> <tr> <td><code>reqform-I</code></td><td></td></tr> <tr> <td><code>reqform-II</code></td><td>Registration requirement form.</td></tr> <tr> <td><code>coadvisor-I</code></td><td></td></tr> <tr> <td><code>coadvisor-II</code></td><td>Coadvisor justification form.</td></tr> <tr> <td><code>boardapproval-I</code></td><td></td></tr> <tr> <td><code>boardapproval-II</code></td><td>Boards approval form.</td></tr> <tr> <td><code>advisorsapproval-I</code></td><td></td></tr> <tr> <td><code>advisorsapproval-II</code></td><td>Advisors approval form.</td></tr> <tr> <td><code>receipts-II</code></td><td>Receipts forms (one per board member).</td></tr> <tr> <td><code>examinersforms-I</code></td><td></td></tr> <tr> <td><code>examinersforms-II</code></td><td>Grades and correction forms (per board member).</td></tr> </table>	<code>reqform-I</code>		<code>reqform-II</code>	Registration requirement form.	<code>coadvisor-I</code>		<code>coadvisor-II</code>	Coadvisor justification form.	<code>boardapproval-I</code>		<code>boardapproval-II</code>	Boards approval form.	<code>advisorsapproval-I</code>		<code>advisorsapproval-II</code>	Advisors approval form.	<code>receipts-II</code>	Receipts forms (one per board member).	<code>examinersforms-I</code>		<code>examinersforms-II</code>	Grades and correction forms (per board member).
<code>reqform-I</code>																							
<code>reqform-II</code>	Registration requirement form.																						
<code>coadvisor-I</code>																							
<code>coadvisor-II</code>	Coadvisor justification form.																						
<code>boardapproval-I</code>																							
<code>boardapproval-II</code>	Boards approval form.																						
<code>advisorsapproval-I</code>																							
<code>advisorsapproval-II</code>	Advisors approval form.																						
<code>receipts-II</code>	Receipts forms (one per board member).																						
<code>examinersforms-I</code>																							
<code>examinersforms-II</code>	Grades and correction forms (per board member).																						

<code>rectifyapproval-I</code>	
<code>rectifyapproval-II</code>	Corrections approval form.
<code>internreqform</code>	Internship Registration requirement form.
<code>internsupervisorform</code>	Internship Supervisor evaluation form.
<code>interntutorform</code>	Internship tutor evaluation form.

Please note that those '-I' regards TCC-I, while '-II' regards TCC-II.

<code>\SetForm</code>	<code>\SetForm {<form-hash>}{<field>}{<code>}</code>
<code>\MakeForm</code>	<code>\MakeForm {<form-hash>}</code>
new: 2023/11/18	<code>\SetForm</code> can be used to set new forms (or redefine existent ones). <code><form-hash></code> being a free identifier. Possible <code><field></code> values are <i>top</i> , <i>heading</i> , <i>bookmark</i> , <i>title</i> , <i>opening</i> , <i>body</i> , <i>closing</i> and <i>footnone</i> . <code>\MakeForm</code> typesets the selected form.

7 UFRGSCCA-LISTS PACKAGE

The following packages are always pre-loaded: *newfloat*, *listings* and *xcolor*. It defines a new *floating environment* *codelist*. Combined with *listings* one can typeset exempts of a *code listing*.

7.1 *Environment*

<i>codelist</i>	<code>\begin{codelist} ... \end{codelist}</code>
	<code>\caption</code> will be named 'Listing' (Listagem).
	LaTeXCode:

```
\begin{codelist}[htbp]
  \caption{sample C code}
  \label{code01}
  \begin{lstlisting}[language=C]
    struct i2c_msg
    {
      __u16 addr;      /* endereco do escravo */
      __u16 flags;
    }
  \end{lstlisting}
  {\sourcecitation{\textcite{Garg:SMA-2000}}}
\end{codelist}
```

7.2 *Declared Commands*

<code>listofcodelist</code>	<code>\listofcodelist</code>
	This will create the 'List of ...' associated with the <i>codelist</i> environment.

\DeclareNewFloat	\DeclareNewFloat {<env-name>} {<file-ext>} {<listname>} {<listofname>}
-------------------------	---

A new float environment, named *env-name*, will be created. Captions will be associated (numbered) as <listname> **num.**. Finally, an associated command `\listof...` will be defined, using <listofname> as a numberless `\chapter` title.

L^AT_EXCode:

```
\def\listingname{Listing}%
\def\listlistingname{List of Listings}%
\DeclareNewFloat{codelist}{lox}{\listingname}{\listlistingname}%%
%% after that, one can do as in the previous example
%%
%% the list of, will be created as
\listofcodelist
```

8 UFRGSCCA-COORD PACKAGE (EXTENDED DOCUMENTATION)

This package defines a set of auxiliary commands meant to support the Professor coordinating students work. it will always pre-load the *longtable* and *ufrgscca-forms* packages.

N.B. It might be also useful to use the commands defined at subsection 6.2, Forms Defined Commands.

A *report document* is composed of 2 main parts:

1. A global preamble, where one sets
 - 1.a. the current semester, Course/TCC/internship coordinator's names, etc. ,
 - 1.b. auxiliary data, like students *check list* items and
 - 1.c. students data.
2. A 'final part' whereas one set which reports are to be generated.

One can (should) use the commands listed at subsection 4.1, Core Commands, and these below:

Check List

<code>\checkdef</code>	<code>\checkdef {⟨LxCy⟩}{⟨check-item⟩}{⟨check-text⟩}</code>
<code>\checklist</code>	<code>\checklist {⟨check-items-list⟩}</code>

updated: 2023/11/18

Whereas one has a '5x5 matrix' (`⟨checkLC⟩` being one of L1C1...L1C5, ... , L5C1...L5C5). `⟨check-item⟩` is a free identifier (to be used with the `\checklist`), and `⟨check-text⟩` the text to appear in the 'check list report'. Note this is a list **per activity** (the current one being set).

`\checklist` set's those items for the current student. `⟨check-items-list⟩` is a comma separated list of `⟨check-item⟩`.

LaTeXCode:

```
\checkdef{L1C1}{tcc-part}{Rel. Parcial} % this creates the '
    check item' tcc-part and associates it with the L1C1 position (first
    line, first column), display text 'Rel. Parcial'
\checkdef{L2C1}{partOK}{Aprov. Rel. Parcial} % this creates 'partOK'
    and associates it with L2C1 position

\checkdef{L1C2}{board}{Banca def.} %
\checkdef{L2C2}{board-date}{Data defesa} % 'board-date' is
    associated with the L2C2 position

\checkdef{L1C5}{tcc-final}{TCC final} %
\checkdef{L2C5}{approval}{Aprovação Correções} %
\checkdef{L4C5}{exam}{Em Exame} % 'exam' (display 'Em
    Exame') is associated with the L4C5 position
%%
%%
%% later on, one can use (inside a \NewStudent command)
\checklist{tcc-part,partOK,exam} % this will, for a
    given student, 'mark' the 'tcc-part', 'partOK' and 'exam' items.
```

Auxiliary / Report Specific

<code>\ActivitySetNewEvent</code>	<code>\ActivitySetNewEvent [⟨act-hash⟩]{⟨event-hash⟩}{⟨event-desc⟩}</code>
<code>\ActivitySetEventDay</code>	<code>\ActivitySetEventDay [⟨act-hash⟩]{⟨event-hash⟩}{⟨day⟩}{⟨week⟩}</code>

new: 2023/11/18

An activity may have a calendar/set of associated events. `⟨event-hash⟩` is just a hash to reference it (*starray* hash). `⟨event-desc⟩` is the text associated with it. `⟨day⟩` and `⟨week⟩` the associated date.

<code>\studentremark</code>	<code>\studentremark {⟨remarks⟩}</code>
<code>\studentnewpage</code>	<code>\studentnewpage [⟨student-hash⟩]</code>
<code>\distinctboard</code>	<code>\distinctboard</code>

new: 2023/11/18

Those commands are only of use when using *ufrgscca-coord*. `\studentremark` sets a free remark text (notes about). Whilst, `\distinctboard` and `\studentnewpage` set the *flag-distinctboard* and *flag-newpage* flags..

<code>\studentCase</code>	<code>\studentCase {<if-A-B-C>} {<if-D>} {<if-Exam-C>} {<if-Exam-D>}</code>
<code>\studentAdvCase</code>	<code>{<if-FF>}</code>
<code>\studentCoadvCase</code>	<code>\studentAdvCase {<if-more-than-one>} {<if-not>}</code>
<code>\studentDismissOtherCase</code>	<code>\studentCoadvCase {<if-defined>} {<if-not>}</code>
<code>\studentNewPageCase</code>	<code>\studentDismissOtherCase {<if-dismiss-other>} {<if-not>}</code>
<code>\studentDistinctBoardCase</code>	<code>\studentNewPageCase {<if-newpage>} {<if-not>}</code>
<code>\studentReviewerCase</code>	<code>\studentDistinctBoardCase {<if-distinct>} {<if-not>}</code>
<code>\studentReviewerSetCase</code>	<code>\studentReviewerCase {<rev-num>} {<if-marked>} {<if-not>}</code>
	<code>\studentReviewerSetCase {<rev-num>} {<if-set>} {<if-not>}</code>
new: 2023/11/18 updated: 2023/12/02 updated: 2024/01/15 updated: 2024/02/22 updated: 2024/02/27	

These are a set of auxiliary conditionals, for instance, `\StudentCase` will execute *only one* of the `<if->` accordingly.

<code>\professor</code>	<code>\professor [<prof-ref>] {<last>} {<first>} {<email>} {<phone>} [<gender>]</code>
<code>\advisorprof</code>	<code>\advisorprof {<prof-ref>}</code>
<code>\coadvisorprof</code>	<code>\coadvisorprof {<prof-ref>}</code>
<code>\examinerprof</code>	<code>\examinerprof {<prof-ref>}</code>
<code>\altexaminerprof</code>	<code>\altexaminerprof {<prof-ref>}</code>

new: 2023/11/30

`\professor` creates a `<prof-ref>` alias to a person's full name. `\advisorprof` expands to a call for `\advisor` and `\advisorinfo` using the properties stored by `\professor`. Similarly, `\coadvisorprof`, `\examinerprof` and `\altexaminerprof` results in call to `\coadvisor`, `\coadvisorinfo`, `\examiner`, `\examinerinfo`, `\altexaminer` and `\altexaminerinfo` respectively.

Note: `\professor` creates/uses an auxiliary *starray*, just mapping names for users convenience.

<code>\boardtitle</code>	<code>\boardtitle {<title>}</code>
<code>\boardobs</code>	<code>\boardobs {<obs>}</code>
<code>\semester</code>	<code>\semester {<semester>}</code>

new: 2023/11/18

`\boardobs` allows to add an observation (`<obs>`) for the 'boards schedule report', and `\semester` sets the current semester value.

Student Specific Commands

<code>\studentfate</code>	<code>\studentfate [<fate>]</code>
---------------------------	--

updated: 2023/11/18
updated: 2024/02/15
updated: 2024/02/25

This assigns the `<fate>` of a student, for those cases that one cannot rely on the 'calculated one' (from examiners individual grades). `<fate>` can be either *exam* (if the student is in exam, but didn't got a grade yet) *C* or *D* (in case a student in exam got graded), *FF* for those that haven't finished the work, *dismiss* for those that, for whatever reason, got dismissed and (finally, 2024/02/25) *other* for all other cases (mainly record keeping). The default is 'do nothing' (no `<fate>` assigned)

Note: (2024/02/15) In case some other, odd, value is assigned, this command will record as if the student is in 'exam', with the given grade `<fate>` marked in bold red.

\studentaddtolist **\studentaddtolist** {<listID>}

new: 2023/12/04

Adds the student to a given list (defined by <listID>), to be later used by **\sortstudentlist** and **\tcreport**.

\checklist **\checklist** {<csv-checkitems>}

<csv-checkitems> is a csv list of valid 'items' (the ones defined by **\checkdef**) and it will 'mark' (check) the corresponding items for a given student.

\timeslot **\timeslot** [<local>] {<date>} {<time>}

updated: 2023/11/29

To set the <local>, <date> and <time> of a student's presentation work.

Note: <date> must be given in numerical form, either <day>/<month> or <day>/<month>/<year>. The day-of-the-week will be obtained using **pgfcalendar**.

Likewise, <time> must be given in a (24h) <hour>:<min> format.

\worktitle **\worktitle** {<title>}

\studentremark **\studentremark** {<remark>}

updated: 2023/11/18

\worktitle sets the current student "work's title". **\studentremark** just inserts a <remark>, which will appear in the *report*'s report.

\distinctboard **\distinctboard**

updated: 2023/11/18

Normally, the default, it's assumed that the student's advisor will also be a member of the student's exam board. For the ones in which this doesn't holds true, one should use the **\distinctboard** after setting a student's name (via **\student**) and before setting its advisor name (via **\advisor**).

For example:

LaTeXCode:

```
\student[Arthur]{last}{first}[m]

\studentinfo[] {243716} {email@somewhere}
\worktitle{work title}
\distinctboard
\advisor{de Amorin}{Heraldo José}[m]
\examiner{Götz}{Marcelo}[m] % He will be the 1st
    examiner
\examiner{Comparsi Laranja}{Rafael Antônio} % the 2nd
\examiner{Ventura Bayan Henriques}{Renato} % the 3rd
```

\examinergrades **\examinersgrades** {<N1>} {<N2>} {<N3>} [<N4>] *

updated: 2023/11/18

updated: 2024/02/15

Quite obvious, this set the grades given by an examiner (the one defined by the 'last' **\examiner** before this.). In case <N4> is given it's assumed the TCC-I case, otherwise TCC-II. (update:2024/02/15) The 'star' at the end will mark said reviewer, in red, when generating a report.

Reports Command

<code>\setstudentlist</code>	<code>\setstudentlist {⟨listID⟩} {⟨list⟩}</code>
------------------------------	--

updated: 2023/11/29

This command will define/create a list named `⟨listID⟩` composed of a csv `⟨list⟩` of student hashes (as defined by `\student [⟨student-hash⟩] {⟨...⟩}`).

<code>\sortstudentlist</code>	<code>\sortstudentlist {⟨listID⟩}</code>
	<code>\sortstudentlist* {⟨listID⟩}</code>
	<code>\sortstudentlist+ {⟨listID⟩}</code>

new: 2023/12/04

These will sort (and classify) a given student list defined by `⟨listID⟩`. The star version sorts the list by student's full name, the *plus* version sorts the list by student's presentation date. By default, the list remains unchanged (no sort).

<code>\tcreports</code>	<code>\tcreports [⟨report-list⟩] {⟨listID⟩}</code>
-------------------------	--

This will typeset the many reports, using the student list defined by `⟨listID⟩`. Where `⟨report-list⟩` is a csv list of keys as follow:

<code>calendar-I</code>	Calendar for the period, TCC-I.
<code>report-I</code>	a student control report, for TCC-I.
<code>checklist-I</code>	a student check list, for TCC-I.
<code>revforms-I</code>	per student reviews forms, TCC-I.
<code>referral-I</code>	per student referral letters, TCC-I.
<code>calendar-II</code>	Calendar for the period, TCC-II.
<code>report-II</code>	a student control report, for TCC-II.
<code>checklist-II</code>	a student check list, for TCC-II.
<code>revforms-II</code>	per student reviews forms, TCC-II.
<code>referral-II</code>	per student referral letters, TCC-II.
<code>boards</code>	exam board dates, TCC-II.
<code>attendance</code>	a simple student's attendance list.
<code>cocertificate</code>	per student coadvisor certificate letter (if any).

<code>\suppresschecklist</code>	<code>\suppresschecklist</code>
---------------------------------	---------------------------------

new: 2024/02/18

This will suppress the “student checklist” on the general report.

9 UFRGSCCA-PPC PACKAGE (EXTENDED DOCUMENTATION)

This contains a set of auxiliary commands to keep track of many *indicators* whilst writing a *PPC document* (which is going to be evaluated based on said *indicators*, though the track of those *indicators* themselves shall not appear in the final version of it). Keep in mind, when considering the use of it: “it works as is” but it hasn't being properly debugged, and it might change “as needed locally”.

The packages *longtable*, *pdfcomment*, *mdframed* and *ufrgscqa-curr* will always be pre-loaded.

9.1 Package Options

<code>showind</code>	(for drafts) it will display the report indicators, of those indicators whose family wasn't set to hide.
<code>indlong</code>	(for drafts) when displaying an indicator, the long version of them will be used.
<code>nocomments</code>	(for drafts) comments (created with the command <code>\comment {< >}</code>) will be suppressed.

9.2 Defined Commands

The next few commands use a finite set of `<status>` which are a pre-defined list of:

<code>tbd</code>	“To Be Done”
<code>done</code>	“Done”
<code>review</code>	“to be reviewed”
<code>attention</code>	Needs Attention
<code>NSA</code>	NSA (portuguese for “do not apply”)
<code>noref</code>	no references
<code>EAD</code>	EAD (portuguese for “distance learning”)
<code>MDi</code>	course ware (portuguese for “didactic material”)
<code>DOCs</code>	other DOCs
<code>default</code>	everything else

<code>\declareindicator</code>	<code>\declareindicator** [<status>] {<fam>} {<ID>} {<text>}</code>
<code>\indicatorDesc</code>	<code>\indicatorDesc {<longdesc>} {<extra>}</code>
<code>\indicatorText</code>	<code>\indicatorText {<text>}</code>

`\declareindicator` is the command to create/define a given “indicator”. `<fam>` set's its *family* group, `<ID>` is the particular ID/term used to reference it (in a family of indicators), `<text>` is a short text describing it (it is the text displayed when using the `\indref` below). `\indicatorDesc` adds a `<longdesc>` (long description) and `<extra>` (extra long description) to a defined `\declareindicator` (it will add those text fields to the “last declared one”). `<longdesc>` will also be displayed when using the `\indref` commands, but only if the `indlong` option was used. The `<extra>` will only be used/displayed with the `\PrintIndicators` command. Finally, `\indicatorText` adds a remark `<text>`, which will be also printed out when using `\lstind` (akin of an explanation/remark field.)

<code>\indsetstatus</code>	<code>\indsetstatus [<status>] {<fam>} {<ID>}</code>
<code>\indsetview</code>	<code>\indsetview {<fam>}</code>
<code>\indsethide</code>	<code>\indsethide {<fam>}</code>

`indsetstatus` sets the `<status>` of a given indicator defined by `<fam>` and `<ID>`. `\indsetview` and `indsethide {<s>}` set the visibility (or not) of a given “family” of indicators, meaning, if those indicators are going to be visible or not (command `\indref`, for instance) if the option `showind` is in use.

`\lstind` `\lstind` [`<seclvl1>`] [`<seclvl2>`] [`<fam>`]

`\lstind` will produce a sectioning like list, `<seclvl1>` defaults to `\section` and `<seclvl2>` defaults to `\subsection`, those indicators marked with an `*` (when creating them) will be issued with `<seclvl1>`, those marked with an `+` will be issued with `<seclvl2>`. The indicator's short text will be the sectioning title, whilst the indicator's 'text' (the one assigned with `indicatorText` will be the sectioning body.)

`\PrintIndicators` `\PrintIndicators` [`<fam>`]

`\PrintIndicators` will produce a “list of contents” like list (with cross reference to all used `\indref` pages). It will either issue a list of all `\declareindicator` or just the ones belonging to `<fam>`. `<fam>` can be a csv list of families. Each entry will be composed by indicator's “family”, “ID”, “short text”, “long text” and “extra description” but not the text issued with `\indicatorText`.

`\helpindicators` `\helpindicators`

This will just prints, middle text, a quick “help text” listing the few main “indicators related command” (to help out those less $\text{\LaTeX 2}_{\epsilon}$ savvy writers.)

`\ifshowind` `\ifshowind` [`<code-ifshow>`] [`<code-ifnot>`]

Just a helping command, based on the package options. If the option `showind` was used, `<code-ifshow>` is executed, otherwise `<code-ifnot>`.

`\textmark` `\textmark` [`<status>`] [`<text>`]
`\comment` `\comment` [`<status>`] [`<title>`] [`<text>`]

Those are annotation, remark commands. The difference being that `\textmark` will just highlight the `<text>` (using `<status>` “format”), whilst `comment` will create a “remark box” (the same used when inserting an indicator's reference, commands below).

N.B.: The command `\comment` is suppressed unless the option `showind` is used.

`\indref` `\indref` [`<<>>`]*> [`<status>`] `fam, ID, comment`
`\indreflst` `\indreflst` [`<<>>`]*> [`<status>`] `fam, IDlist, comment`

`\indref` creates a box (*TikZ* based *mdframed*) of the indicator denoted by `<fam>` and `<ID>`. The family and IDs will be issued as the “frame title”, the current indicator's `<status>` will be printed out (the whole box will be highlighted accordly), the short version of the indicator will be used (the long version will “appear” as a *pdfcomment*), finally any `<comment>` will be added to the text box. Each `\indref` box will have a link to the indicator's list (issued with `\PrintIndicators`). If the optional argument `<status>` is used, the indicator's status will be updated accordly. The star version also prints the indicator's long text.

`\indreflst` behaves similarly, with the difference that `<IDlist>` is a csv list of IDs (same family), moreover, each item of said list can have the form either just `<ID>` or `<status:ID>`, in the last form, that ID will have its status updated, as well.

<code>\fancyquote</code>	<code>\fancyquote [$\langle\text{vspc}\rangle$] $\{\langle\text{text}\rangle\}$ $\{\langle\text{author}\rangle\}$ $\{\langle\text{dateref}\rangle\}$</code>
--------------------------	---

As quick “quote” hack, `\fancyquote` will typesets a $\langle\text{text}\rangle$ (small size, italic text, in a minipage environment) followed by $\langle\text{author}\rangle$ and $\langle\text{dateref}\rangle$. This is meant to be used after a `\chapter` or `\section` commands. $\langle\text{vspc}\rangle$ is to be used in case one has to adjust the vertical space between the sectioning command, and the quote one.

<code>\labelhack</code>	<code>\labelhack $\{\langle\text{text}\rangle\}$</code>
-------------------------	--

As the name implies, it is a hack. In some cases (which we haven’t manage to found why/what), `hyperref` would fail miserably when using the `\nameref` (in some cases getting the sectioning correct, but not the name!). This just assures that `\nameref` will use the correct sectioning name in those cases.

For Example:

```
\section{this section}\labelhack{this section}\label{somelabel}
```

<code>\acrodef</code>	<code>\acrodef $\{\langle\text{acroID}\rangle\}$acronymlong</code>
<code>\acro</code>	<code>\acro $\{\langle\text{acroID}\rangle\}$</code>
<code>\acrol</code>	<code>\acrol $\{\langle\text{acroID}\rangle\}$</code>
<code>\acrols</code>	<code>\acrols $\{\langle\text{acroID}\rangle\}$</code>
<code>\acrosl</code>	<code>\acrosl $\{\langle\text{acroID}\rangle\}$</code>
<code>\acrofoot</code>	<code>\acrofoot $\{\langle\text{acroID}\rangle\}$</code>
<code>\printacrolist</code>	<code>\printacrolist [$\langle\text{enumkeys}\rangle$] $\{\langle\text{widest}\rangle\}$</code>

Those are yet another acronym hack. `\acrodef` “creates” an acronym, identified by $\langle\text{acroID}\rangle$, whose short (acronym) version is $\langle\text{acronym}\rangle$ and the long version in $\langle\text{long}\rangle$. `\acro` just typesets the $\langle\text{acronym}\rangle$, `\acrol` the $\langle\text{long}\rangle$ version. `\acrols` typesets the the long version followed by the short (using a comma as separator). `\acrosl` prints the short version first. Finally, `\acrofoot` typesets the short version in text and the long as a footnote. `\printacrolist` creates a description list based on the `listofabbrv` environment.

9.3 Environments

<code>ppc.quote</code>	<code>\begin{ppc.quote} ... \end{ppc.quote}</code>
------------------------	--

This is just a tailored “quote” environment, using almost all page width, just in a smaller font size.

10 UFRGSCCA-CURR PACKAGE (EXTENDED DOCUMENTATION)

The background of it: To have the ability to “describe” (store the information in a “structured way”) an University Course Curricula and have the possibility, later, to presented that same information in many different

ways (including a dependence graph). All data is captured/stored in a set of 3 *stararrays*:

topics' Structure Definition:

```
{
  topichash = ,
  name = ,
  color = ,
  class . struct = {
    classcode = ,
  }
}
```

Classes' Structure Definition:

```
{
  cred = ,
  classcode = ,
  name = ,
  summary = ,
  topichash = ,
  remark = ,
  bib = ,
  bib basic = ,
  bib compl = ,
  ref . struct = {
    currhash = ,
    semhash = ,
    kind = ,
  } ,
}
```

Curricula's Structure Definition:

```
{
  currhash = ,
  name = ,
  text = ,
  sem.struct = {
    pos = ,
    semhash = ,
    name = ,
    class . struct = {
      classcode = ,
      kind = ,
      obs = ,
      pos = ,
      color = ,
      prereqset . struct = {
        prereq . struct = {
          starred = ,
          classcode = ,
          angIN = ,
          angOUT = ,
        }
      } ,
    } ,
  } ,
}
```

The following commands “describe” a curricula, whereas one is a sequence of semesters $\langle \text{semID} \rangle$, each semester is composed by a list of classes, $\langle \text{classID} \rangle$, and each class has a list of dependencies, $\langle \text{classID} \rangle$ as `\depdef`. All those lists are stored as csv lists, so “processing them” can be systematized.

<code>\topicdef</code>	<code>\topicdef</code> [$\langle \text{color} \rangle$] { $\langle \text{topicID} \rangle$ } { $\langle \text{text} \rangle$ }
<code>\defaulttopic</code>	<code>\defaulttopic</code> { $\langle \text{topicID} \rangle$ }

`\topicdef` defines $\langle \text{topicID} \rangle$ (to be used when describing a class) and associates a $\langle \text{text} \rangle$ description and a $\langle \text{color} \rangle$ (for topic highlight). `\defaulttopic` sets the default one (if not explicitly given when describing a class).

<code>\classdef</code>	<code>\classdef [<topicID>] {<classID>} {<cred>} {<name>}</code>
<code>\classset</code>	<code>\classset {<classID>}</code>
updated: 2023/11/18	<code>\classdef</code> defines a class <code><classID></code> (with a given <code><name></code> and number of <code><name></code>) associated with a given <code><topicID></code>. <code>\classset</code> sets the current class. The following commands always refer to the “last defined” <code>\classdef</code> unless <code>\setclass</code> is used, which changes the “current class” for the following commands.
<code>\csummary</code>	<code>\csummary {<desc>}</code>
<code>\classremark</code>	<code>\classremark {<remark>}</code>
<code>\bibdef</code>	<code>\bibdef [<type>] {<bibref>}</code>
new: 2023/11/18 updated: 2025/09/18	<code>\csummary</code> sets a class summary, whilst <code>\classremark</code> annotates a ‘note/remark’. <code>\bibdef</code> is used to set a list of bibliographies, one per issued command. The default <code><type></code> value is <i>main</i>, other possible values <i>basic</i> and <i>compl</i>.
<code>\currdef</code>	<code>\currdef {<curr-ID>} {<short name>} {<long name>}</code>
<code>\semdef</code>	<code>\semdef {<semID>} {<name>} {<pos>}</code>
updated: 2023/11/18	<code>\currdef</code> creates a curricula (with a set of semesters defined as following). <code>\semdef</code> creates a semester, <code><semID></code>, <code><pos></code> being a position ‘hint’ when creating a dependency graph (see below).
<code>\addclass</code>	<code>\addclass [<color>] {<pos>} {<classID>} {<kind>} [<obs>]</code>
<code>\depdef</code>	<code>\depdef* [<pos>] {<classID>}</code>
<code>\altdep</code>	<code>\altdep</code>
updated: 2023/11/18	<code>\addclass</code> adds a class, <code><classID></code>, to the current semester. <code><pos></code> will set the vertical position of this class (when using <code>\GraphEtp</code>. <code><kind></code> will set the class kind and <code><obs></code> will add an observation. <code>\depdef</code> inserts/creates a “class dependency” list. <code><pos></code> is used as a ‘hint’ for the incident (dependency) line angle. The starred version is meant to be used when the ‘dependency’ isn’t another class but rather, for instance, a number of credits. <code>\altdep</code> defines/start and alternate dependency list.
<code>\TabEtp</code>	<code>\TabEtp* [<sectioning>] {<semID>} [<font-fmt>]</code>
<code>\TabTopic</code>	<code>\TabTopic {<topicID>}</code>
updated: 2023/11/18	<code>\TabEtp</code> will construct a longtable with all classes associated with <code><semID></code>. The default ‘font size’, <code><font-fmt></code>, is <code>\footnotesize</code>. The default <code><sectioning></code> is <code>\notoc\section</code>. The non-star version includes the bibliography lists as well. <code>\TabTopic</code> will construct a longtable with all classes associated with a <code><topicID></code>.
<code>\GraphEtp</code>	<code>\GraphEtp {<semID>}</code>
updated: 2023/11/18	It will create a dependency graph for a given <code><semID></code>. N.B. to start with, it is highly dependent on the semester sequence, one shall start with first semester and go from there.